EX15 - NumPy Trajectory - Arrays, Shape and Columns

Beginner Step-by-Step Guide

This guide follows the real file `ex15_numpy_trajectory_analysis.py`. It explains the problem first and then the code from the perspective of someone starting Python.

1. What we want to solve

Represent the trajectory as a NumPy array with shape (N,3) and operate on full columns.

Problem variables / elements

Name	Meaning
trajectory_array	matrix [h,v,t]
energy	g*h + 0.5*v**2

Equation or logic on paper

```
row = [h, v, t]
energy = g*A[:,0] + 0.5*A[:,1]**2
```

Manual check

The first state [[0,50,0]] has shape (1,3). A vector [0,50,0] without double brackets would have shape (3,).

2. What we already know and what is new

Already seen:

- lists
- tuples
- dictionaries
- simulation

New in this exercise:

- np.array
- np.empty((0,3))
- shape
- (3,) vs (1,3)
- np.append with axis=0
- slicing [:,0]
- np.max
- vectorization

3. New Python concepts

1. NumPy array

Syntax: `np.array([[h, v, t]])`

A NumPy array stores numeric data with a shape. Double brackets create a 2D row.

2. shape

Syntax: `A.shape`

shape describes the length of each axis. (N,3) means N rows and 3 columns.

3. axis=0

Syntax: `np.append(A, row, axis=0)`

Here it means rows are appended while keeping the three columns.

4. Slicing

Syntax: `A[:, 0]`

: selects all rows and 0 selects the first column.

5. Vectorization

Syntax: `g*A[:,0] + 0.5*A[:,1]**2`

NumPy applies operations to all column elements without an explicit Python for loop.

4. Code step by step

Step 1. Create empty matrix

```
trajectory_array = np.empty((0, 3))
```

It has zero rows and three columns, ready for [h,v,t].

Step 2. Create first row

```
state_0 = np.array([[config["h0"], config["v0"], config["t0"]]])
```

Double brackets force shape (1,3).

Step 3. Append row

```
trajectory_array = np.append(trajectory_array, state_0, axis=0)
```

np.append returns a new array; therefore we assign the result back.

Step 4. Select columns

```
trajectory_array[:, 0]
trajectory_array[:, 1]
trajectory_array[:, 2]
```

The columns represent height, velocity and time.

Step 5. Compute energy

```
energy = config["g"] * trajectory_array[:, 0] + \
         0.5 * trajectory_array[:, 1]**2
```

The expression is evaluated element-wise and produces one energy value per state.

5. Execution flow

```
generate states
 -> append each [h,v,t] row
 -> matrix (N,3)
 -> slice columns
 -> vectorized analysis
```

6. Expected result / check

The maximum of column 0 and final values of columns 1/2 should match the list-based solution.

7. Common beginner mistakes

- Creating np.array([h,v,t]) and expecting (1,3).
- Forgetting axis=0.
- Thinking np.append modifies in-place.
- Confusing row 0 with column 0.

8. If you remember only five things

- NumPy works with arrays and shapes.
- (3,) is not (1,3).
- A[:,0] means all rows of column 0.
- np.append returns a new array.
- Vectorization operates on many values at once.

9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.